Why Code?

Before supermarket scanners, a cashier would pick up a box of cereal, find its price, and type it into the register.

Then comes the next item. And the next.

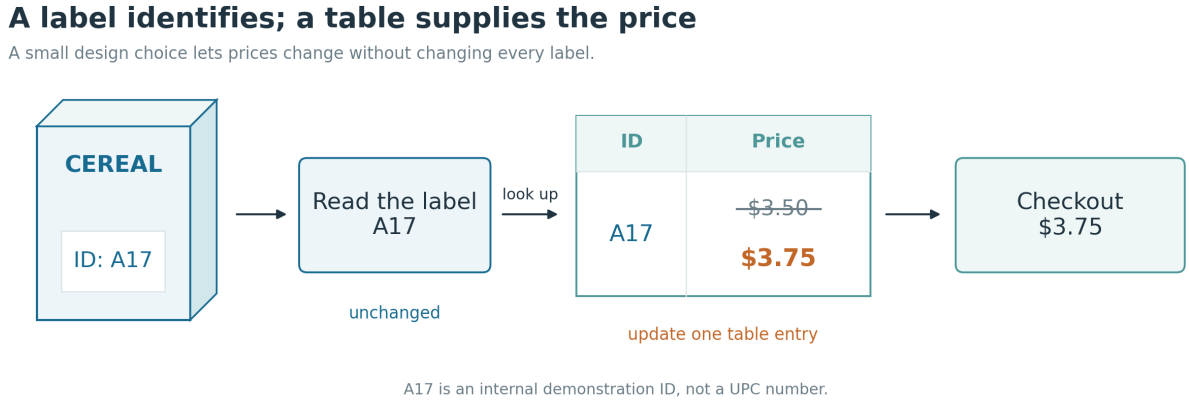
Every item took time. A mistyped price took more time to correct, while the queue grew behind the customer.

What if passing the box over the counter were enough for the machine to recognize it? The cashier could keep the groceries moving, without stopping to look up and type each price.

That question helped set barcode development in motion. Woodland and Silver explored automatic product identification in the late 1940s. Decades later, George Laurer's team at IBM developed the UPC symbol. In 1974, a pack of chewing gum became the first item scanned with a UPC at a retail checkout. Historical account: IBM.

Suppose we give our cereal box a label the machine can read. What should it say?

Printing the price seems tempting. But what happens when the cereal goes on sale? Instead, we can give the product a stable number and let the store look up its price.

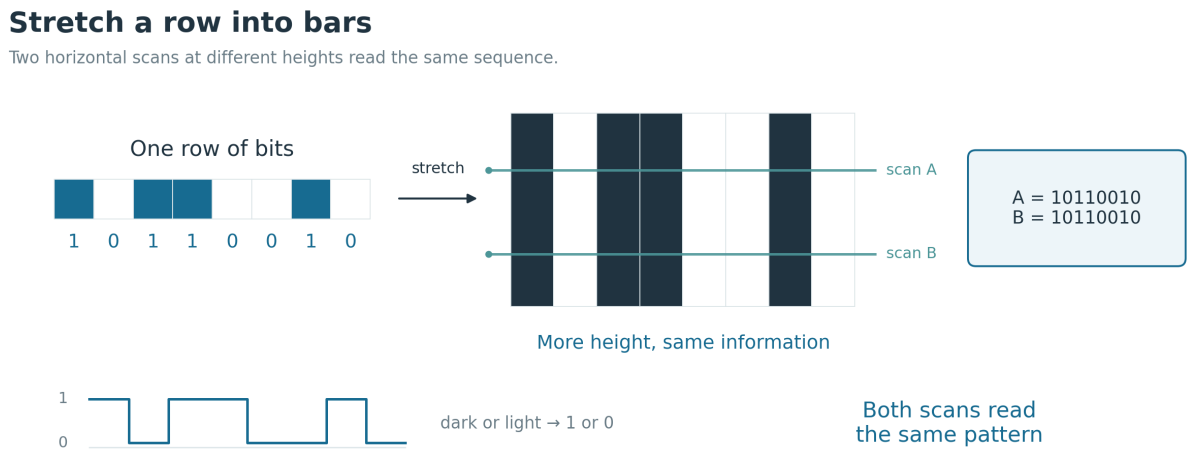


The product stays the same. Its price can change.

Now we need to get that number from the box into the machine.

We can give an optical reader a simple task: distinguish light from dark as it crosses the label. Arrange black and white patches in an agreed pattern, and their changing brightness becomes a signal.

Try printing a thin row of little squares. A scan passing just above that row misses it. Make the squares taller, and a scan can cross higher or lower while meeting the same pattern.



Taller bars let the reader scan at different heights and still read the same sequence of 1s and 0s.

The bars can now stand for digits. The printer and reader must agree on which pattern represents each digit: this pattern means 0, that pattern means 1, and so on. They also need signs that help separate the code from its surroundings and organize the digits inside it.

Look at a UPC-A barcode with those pieces picked out:



The end markers show where the barcode begins and ends; the middle marker separates the two groups of digits. The final digit helps detect reading errors.

There is still a problem. What if the reader gets a digit wrong?

We can make the last digit depend on all the others. For UPC-A, multiply the first eleven digits alternately by 3 and 1, starting from the left, then add those products. Choose one final digit to add to that sum so the total is a multiple of 10. **Any total ending in 0 passes this check.**

For our example, the first eleven digits give

$$3(0+6+0+2+1+5)+(3+0+0+9+4)=58.$$

So we append the check digit 2. The total is now $58 + 2 = 60$, which ends in 0.

Now change one digit.

The check detects an error. It does not tell us where.

Multiply, then add. For a valid number, the total must end in 0.

multiply by	3	1	3	1	3	1	3	1	3	1	3	1		
original	0	3	6	0	0	0	2	9	1	4	5	2	→ total: 60	Ends in 0 check passes
change position 2	0	**4**	6	0	0	0	2	9	1	4	5	2	→ total: 61	Ends in 1 ERROR DETECTED
change position 4	0	3	6	**1**	0	0	2	9	1	4	5	2	→ total: 61	Ends in 1 ERROR DETECTED

Both errors are detected; the check does not identify their locations.

Both errors are detected because 61 does not end in 0. The check alone does not tell us which digit was read incorrectly.

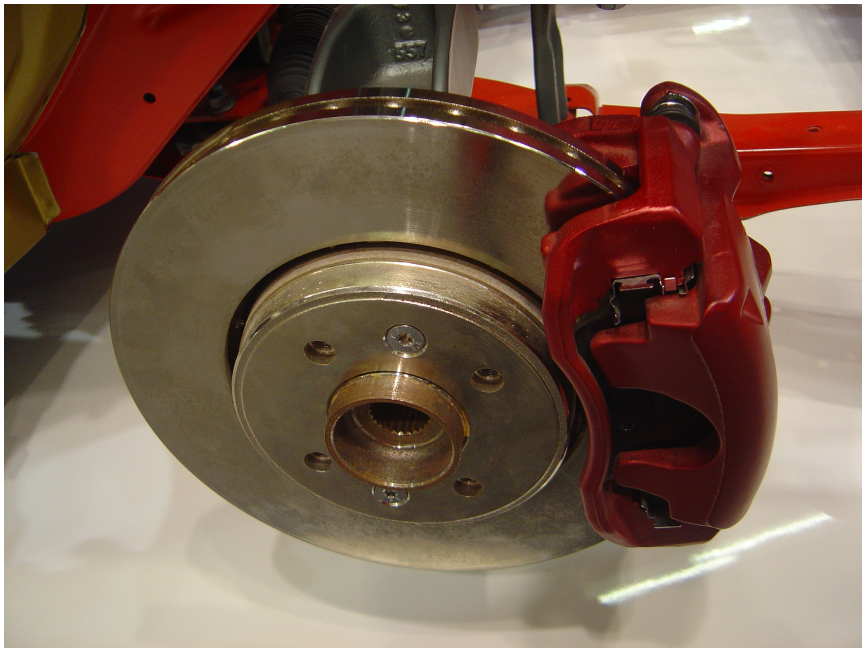
If the check detects an error, the checkout can ask for another scan. Once the number has been read and checked, the computer looks up the product, displays its price, and adds it to the bill.



What could we do with more check digits, connected to the data in more elaborate ways?

Move the same idea to a factory.

A car's brake assembly travels through several stages of production. Information about its identity, production batch, and inspection needs to travel with it. Put that information into several barcodes, and someone has to scan them one after another.

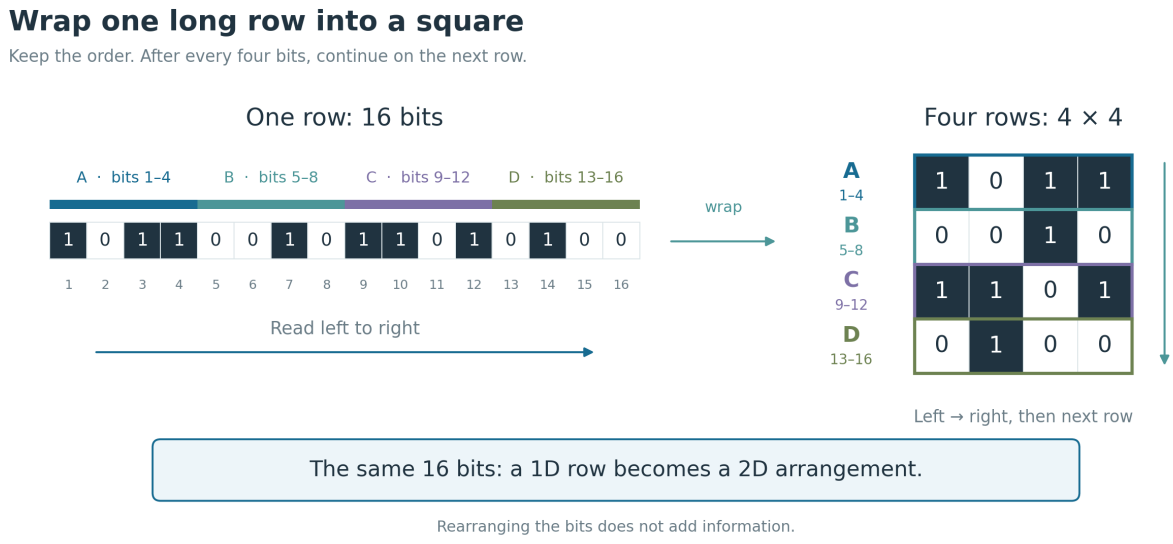


By the early 1990s, DENSO's engineers were hearing requests for faster reading while manufacturing sites needed more information on smaller labels. Masahiro Hara's team developed QR Code, released in 1994, in response to those needs. Historical account: DENSO WAVE.

Look back at the tall bars. Every horizontal row repeats what the row above already says.

What if the next row could say something different?

Take a single row of sixteen squares and wrap it into four rows of four.



The same sixteen bits, arranged in two dimensions. Each square now has a row and a column.

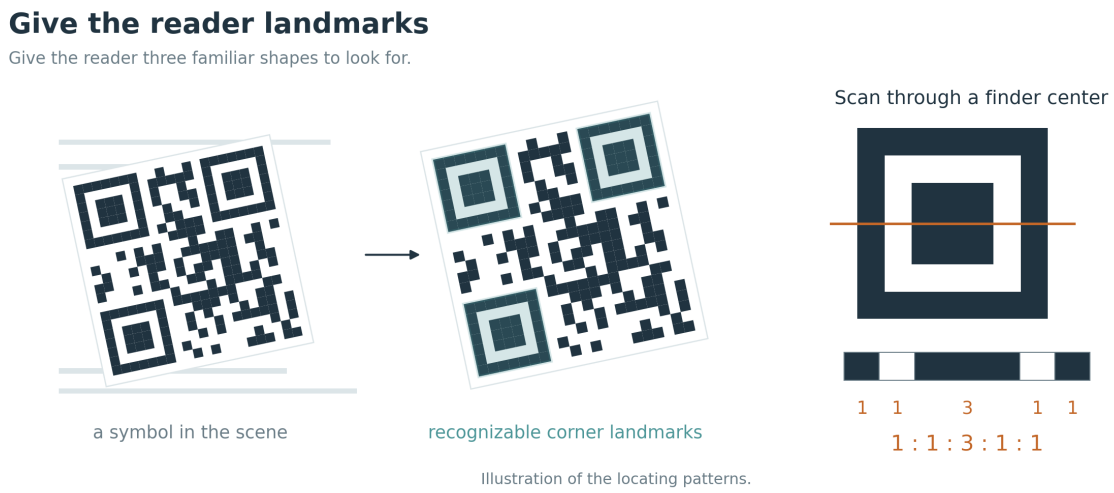
A one-dimensional barcode changes its pattern along one direction. Here, the pattern can change across and down. The same data fit into a compact square, which a reader can capture as an image.

Put that little square on a busy package, though, and a new problem appears. Where does the code begin? Which way is it facing? Which black marks belong to the code?



We could try reading it in several orientations, but that adds work when our goal is a quick scan. We need a recognizable reference that tells the reader which way the code faces, so it can follow the agreed reading order.

The three large corner patterns in a QR Code help the reader find the symbol and its orientation. A line through the middle of one has a distinctive rhythm: black, white, a wider black center, white, black.



The 1:1:3:1:1 proportions stay the same as the pattern gets larger or smaller.

DENSO's developers examined printed material when choosing the proportions 1:1:3:1:1, looking for a

pattern unlikely to be confused with the surrounding print.

Now imagine that part of the label gets dirty or torn.



The reader may find the code perfectly well and still read some of its data incorrectly. We can reserve some of the squares for extra information that helps us recover from mistakes.

Here is a small picture of how.

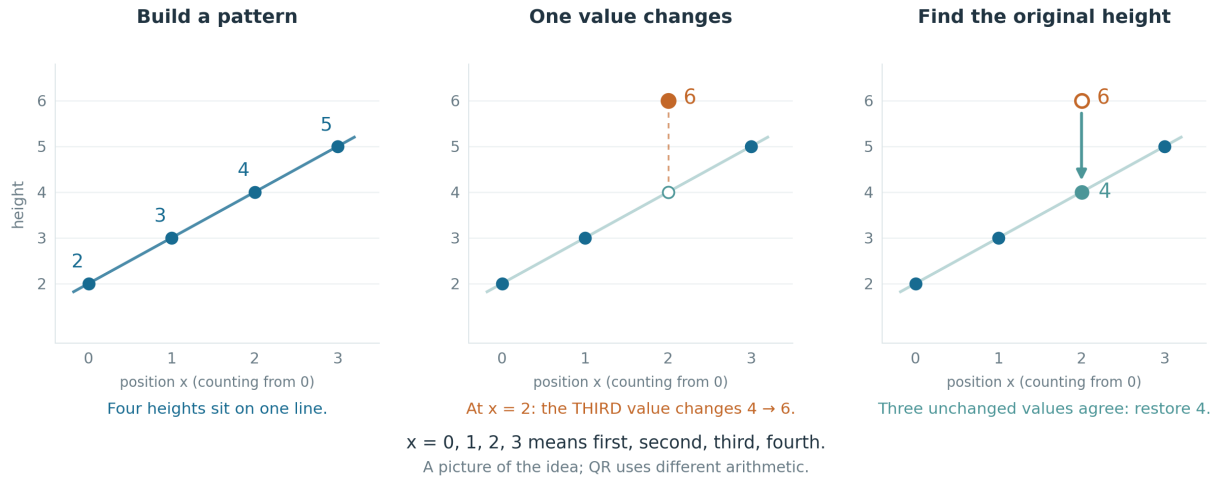
Choose two numbers: **start at 2, then add 1 at each step**.

They describe a straight line. At four agreed positions, its heights are

$$2, \quad 3, \quad 4, \quad 5.$$

With a starting value $s = 2$ and a step of 1, these are simply $s,\ s+1,\ s+2,\ s+3$. Two numbers determine all four heights.

Suppose we send all four heights, and **at most one is read incorrectly**. The positions are numbered 0, 1, 2, and 3. In this example, the third value, at position 2, is read as 6 instead of 4.



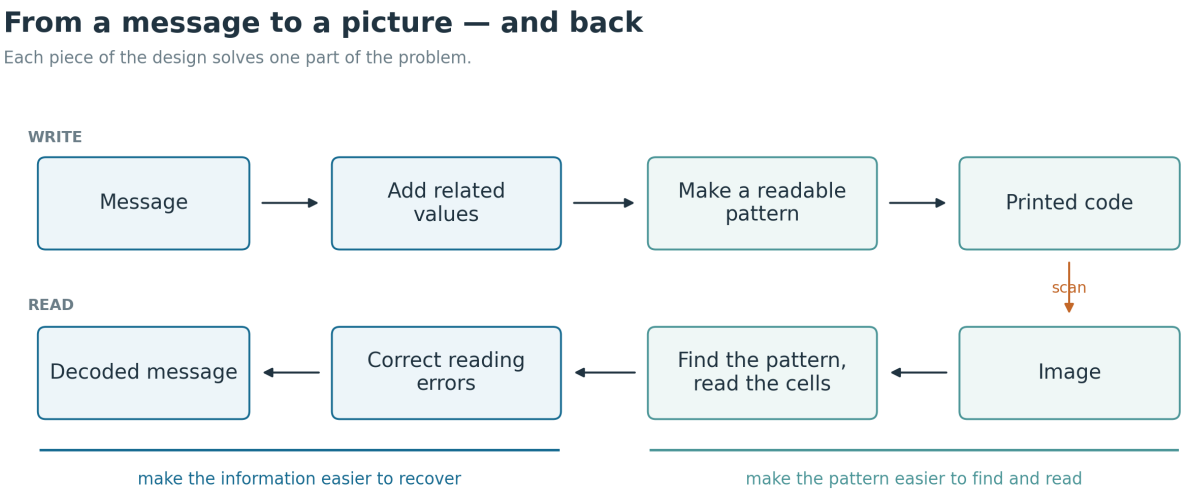
The positions are fixed; one recorded height changes. These points represent numbers, not locations of black squares in a QR image.

To recover the message, look for a straight line through at least three of the received points. The heights at positions 0, 1, and 3 still lie on one straight line. At position 2, that line predicts a height of 4, while the received value is 6. Since we allow at most one error, we can restore 4 there and read off the original message: start at 2, increase by 1.

The extra numbers gave us overlapping clues. Losing agreement at one point did not erase the relationship shared by the others.

This is an entry into the idea behind Reed–Solomon coding, which QR uses to protect its data. The actual code works with groups of bits and different arithmetic. Adding related information leaves us more clues. After an error, enough of those relationships may survive to reconstruct what was sent.

The printed pattern carries information in two directions. The locating patterns help the reader find it. Extra relationships among the data help repair some mistakes made while reading it.



Follow the message into the printed pattern, then follow it back out.

How much extra information should we add? Could we recover two wrong values? At what point would two different original messages look equally plausible?

Those questions lead from the pictures to the mathematics of codes.

And the data need not be printed. A message crossing a network or a value stored in memory can also be corrupted. What relationships could we build into the data so that enough evidence survives a mistake?